RSI、反馈与学习动力学

价值、认识与学习机制的自我修改｜文献综述

递归自我改进（RSI）关心的是：系统学到的能力，能否帮助它改进下一轮学习？例如，会写代码的系统可以修改自己的工具，也可以修改负责生成改进方案的程序。两者都属于自修改，但改变的环节不同。本文按价值、认识和学习机制梳理这些研究，再讨论同一反馈循环为何既可能推动进步，也可能维持错误。

类别	修改什么	需要回答的问题
价值与评价	目标、训练奖励、评分规则	凭什么判断修改更好?
认识与交互	工具、实验选择、世界模型	怎样获得并理解纠错证据?
学习机制	更新规则、架构、记忆、改进程序	怎样让下一轮更会学习?

一、价值与评价：先区分训练奖励与最终目标

奖励是训练时给学习者的信号，最终目标是评价学习结果的标准。Singh 等说明，两者可以不同：只奖励眼前收益，可能学不到以后有用的探索行为；合适的内部奖励反而能提高长期收益。价值函数则是在给定奖励下估计预期回报。改进估计、改写奖励、改变目标，是三个不同问题。

固定最终目标时，奖励设计可写成下面的双层优化。先按奖励 r 训练出策略 A(r)，再用外部标准 J 评价它；选择的是训练效果最好的奖励：

$$r^{*} \in \operatorname{argmax}_{r} E[J(A(r))]$$

期望表示对训练随机性取平均。Eureka 把这一思路做成循环：语言模型写奖励代码，强化学习算法据此训练策略，再把训练结果交回语言模型修改奖励。作者比较一次生成更多候选与分轮使用反馈，也检查去掉各奖励项的训练统计会怎样。结果支持反馈能帮助奖励搜索。在固定任务目标下，它改进的是训练手段。

目标本身改变时，需要说明按谁的标准评价

Everitt 等研究效用函数也可被修改的主体。若用修改后的标准评价修改，主体可能通过把所有结果都判为“好”来获得高效用。若沿用当前标准，并预测修改会怎样改变未来行为，修改的激励就不同。论文在信念和初始效用不直接依赖修改、初始策略最优等条件下证明：按当前标准预测修改后行为的规则，可以保持实际行动对原目标的最优性。

决策论把“是否值得修改”也纳入选择

修改和计算都要付出成本。Hay 等研究是否值得再做一次模拟：只有它对最终决策的预期帮助超过成本，才值得继续。Gödel machine 把判断扩展到程序自修改，包括修改证明搜索器；系统先证明现在修改优于继续搜索，再执行修改。这个保证依赖公理和可证明性。现代 RSI 多用任务实验筛选修改，因此下文还需区分证明保证与实验依据。

二、认识与交互：获得证据，还要能够利用证据

选择信息，要看它能否改善行动

目标确定后，学习者还要决定去看什么、试什么。Blackwell 比较两种信息来源：如果只根据信号本身、按固定规则删减或加噪，就能生成另一种信号，那么在目标相同、不计获取成本且能最优使用信息时，保留详细信号不会降低决策价值。这里衡量信息的依据，是它能帮助作出什么选择。

双重控制进一步让行动承担两种作用：完成当前任务，同时试探未知规律。控制实验改变探索成本和未知参数对任务的重要性，检验何时值得为未来学习牺牲眼前收益。信息导向采样（IDS）则把探索指向“哪个行动最好”。它权衡当前选错行动的预期损失与关于最优行动的信息增益；无需同等重视世界中所有未知细节。

相同的信息，对不同学习者未必同样有用

上述决策分析还需要补上一个问题：学习者能否读懂拿到的信息？Xu 等把可用的预测模型限制在集合 $\mathcal{V}$ 中，用这组模型能达到的最低预测损失定义条件 V-熵：

$$H_{\mathcal{V}}(Y|X) = \inf_{f \in \mathcal{V}} E[-\log f(X)(Y)]$$

X 是输入，Y 是要预测的结果，f(X)(Y) 是模型分给该结果的概率。比较“不看 X”和“看 X”时的最低损失，就得到输入对这类模型的可用信息。只要允许模型忽略输入，这个差值就非负。仅对输入做确定性处理，不会增加它与预测目标的 Shannon 互信息，却可能让有限模型更容易识别其中的规律。

Abbe 等研究学会简单特征怎样帮助学习复杂特征。例如，目标中包含 x_1、x_1x_2、$x_1x_2x_3$，就提供了逐级扩展的起点。其“合并阶梯”条件把这种结构一般化：乘积项可以排序，每项最多引入一个新变量。若只有孤立的高阶乘积项，就缺少这种低阶学习信号。相关结果针对均匀比特输入、稀疏目标与宽两层网络，使用平均场极限及随输入维数同阶增长的样本规模。它研究的是给定算法能怎样学到结构。

已经学到多少，也可以影响下一步去学什么

Schmidhuber 用压缩进展选择经验：在同一段历史上，新压缩器比旧压缩器省下多少编码长度，就奖励多少。奖励来自实际学到的规律，而非数据仅仅看起来新奇；更新算法可以保持固定。由此，过去的学习结果开始影响未来的经验选择。

Finzi 等的 epiplexity 关注另一个对象：有限计算时间内，数据中有多少结构可被学到。它把描述数据的编码分为两部分：一部分描述模型学到的规律，另一部分记录仍无法预测的内容。在计算时间受限时，选择总编码最短的模型；epiplexity 衡量其中描述规律的编码长度。棋局预训练实验显示，这种结构对不同下游任务的帮助并不相同。因此，“数据有可学结构”“学习已经取得进展”和“任务表现改善”需要分别判断。

认识的自修改：工具、技能与世界模型

现代系统通过不同方式改变与世界的交互。Toolformer 学会使用既有工具，Voyager 把学到的技能保存成新工具，Memento 3 则修改用来预测行动后果的世界模型。三者分别回答怎样调用、怎样复用、怎样理解。

工具与技能：让已有能力成为下一步的资源

Toolformer 让语言模型尝试调用计算器等工具。系统只保留能降低后续文本预测误差的调用样本，再用这些样本训练模型。作者比较同语料下不学工具调用的模型，也检查推理时关掉工具会怎样。计算器带来的额外收益说明，模型学到的调用方式确实有用。被改变的是何时、怎样调用，工具集合和样本筛选规则仍由人给定。

Voyager 在游戏中生成执行动作的代码，根据环境反馈反复修正，再把成功代码存入技能库。后续任务可以直接调用或组合旧技能。移除技能库后，探索更早停滞；新世界任务进一步检验了技能复用。它展示了知识积累如何帮助后续学习。这里增长的是技能内容，选题、检索和自验证规则仍预先设定。

世界模型：用过去的经验预测尚未做过的事

Memento 3 把经验写成描述环境规律的“规则书”，再转成可运行的世界模型。系统先在模型中预测行动后果，再去环境执行，并根据新反馈修订规则。在删除规则书的对照中，两种系统都能完成所选游戏，但保留规则书需要的环境动作更少。这个比较说明，组织和修订经验能够改善下一轮交互。

世界模型还必须接受纠错。Gelman–Shalizi 区分两种工作：在已有模型中更新信念，以及发现模型无法解释数据后修改模型本身。这个区分也说明了 Memento 的验证边界：重现已有经历可以发现部分错误，但多个模型仍可能解释同一历史，对未试过的行动却给出不同预测。要区分它们，还需要新的交互。

三、学习机制：从学习知识到学习更新规则

工具和世界模型改变了系统能做什么；元学习进一步改变系统怎样学。学习型优化器根据梯度历史生成参数更新，架构搜索学习提出哪些网络结构，元强化学习则把交互历史存入循环网络状态，据此适应新任务。最后一种方法在测试时保持网络权重不变，适应发生在内部状态中。

这些工作分别用传统优化器、随机搜索或既有学习规则作比较，检验学得的机制是否有效。但训练这些机制的外层算法仍由人设定。由此引出下一步：负责产生改进的程序，能否也成为系统自己的修改对象？

学习机制的自修改：改进程序也能被改进

STOP 与 DGM：让修改后的程序参与下一轮

找到更好的任务程序，与更会修改程序，需要分开评价。STOP 让一个代码改进程序修改自身，再让新版本去改进一组下游程序，以后者的平均任务表现评价新版本。与初始改进程序、单次推理和贪心迭代相比，部分模型取得收益，较弱模型也可能退化。它把“产生改进的能力”变成了直接优化的对象。

Darwin Gödel Machine（DGM）让修改后的程序既解决编程任务，也负责生成下一轮修改，并保存多个历史版本。作者设置两种对照：始终由最初程序负责修改；只沿最新可用版本继续修改。始终由最初程序负责修改时，后续改善较早趋缓；只保留最新版本时，则更容易被较差修改阻断。比较支持两点：新能力参与后续修改有帮助，历史分支也能保留继续探索的机会。

HyperAgents：进一步修改产生改进方案的过程

DGM 仍由固定的外层流程生成修改指令。HyperAgents 把任务程序和负责修改它的程序一起开放给系统编辑，允许错误诊断、方案生成和后续修改的组织方式发生变化。DGM 依靠编程能力同时支持解题和自修改；HyperAgents 进一步检验这类改进能否迁移到其他任务。两者的推进关系在于可修改范围和迁移证据。

为评价改进能力，作者在测试时固定改进程序，让它从给定任务程序出发生成候选，再用验证集选优、测试集评价。迁移来的改进程序在新任务上带来更大改善。但迁移也带入了任务代码；数学评分的初始失败还包含输出格式错误。因此，这些实验支持整个系统的跨任务迁移，尚未完全分离改进程序自身的贡献。

自修改的效果，需要实验检验

Gödel machine 要求先证明修改有益；DGM 等系统先尝试修改，再用任务实验判断。历史版本即使暂时较弱，也可能被保留，供以后继续修改。基础模型、外部评价和部分版本选择规则仍固定。这使递归改进可以在有限任务上检验，其有效范围也由这些实验条件决定。

持续学习补上一个条件：系统仍需学得动

Dohare 等考察持续训练后的网络还能否学好新任务，这种能力称为“可塑性”。实验发现，常规网络可能逐渐失去可塑性；重置低效用单元可以缓解问题。这与忘掉旧知识是不同的现象。该方法的重置规则由人设定，但它揭示了 RSI 所需的条件：即使新反馈不断到来，系统仍要保有吸收它的能力。

四、反馈怎样支持或限制自我改进

Berk–Nash 解释行动与证据怎样相互支持

反馈能否纠正当前判断，是这些研究的共同问题。Berk–Nash 允许主观模型类错设：行动决定哪些世界回复出现，信念只支持最符合这些回复的模型，实际行动又在该信念下最优。最佳拟合集合由行动频率加权的预测误差决定：

$$\Theta^*(\rho) = \operatorname{argmin}_\theta \; E_{a\sim\rho} \; KL(Q_W(\cdot|a) \;\|\; Q_\theta(\cdot|a))$$

ρ 是行动频率，θ 是主观模型参数；两个 Q 分别表示真实与主观反馈分布，KL 衡量预测差异。信念只给 Θ* 中的模型分配概率，按这些概率计算预期收益；实际采用的行动须使该收益最大。拟合与行动两个条件共同维持这个状态。

自我确认均衡突出未尝试行动上的错误；Berk–Nash 还允许模型无法正确解释已见反馈。扩大探索与修正模型类因此解决不同问题。均衡也不保证收敛：Fudenberg–Lanzani–Strack 证明，在其固定模型类的贝叶斯设定下，若行动收敛，该行动须在所有只支持最佳拟合模型的信念下最优。

GAN 与模型崩溃考察反馈的分辨能力与来源

GAN 的有限判别器理论追问：真实样本已经拿到，评价器能否读出其中的差异？Arora 等表明，缺少大量多样性的生成分布仍可能骗过受限判别器。奖励过优化则显示，持续优化代理评分会扩大它与参考评分的分歧。反馈依据存在，还需要足够的分辨能力。

模型崩溃追问：下一轮数据由谁产生？上一代模型若成为新数据源，采样遗漏与拟合误差会进入后代。数据累积研究通过保留原始与历代数据改变这条传递路径。这与 Berk–Nash 的真实但选择性反馈、GAN 的真实样本参照形成比较：反馈来源、证据覆盖与分辨能力共同影响纠错，各项结论仍依赖各自设定。

棋类与代码为何能利用自生成经验

棋类和代码的反馈分析给出两个具体例子。AlphaZero 自己选择对局，但胜负由规则结算；网络引导搜索，搜索与对局结果又训练网络。CodeRL 自己生成程序，但编译和测试给出执行结果，评价网络再把结果转成训练信号。自己产生候选，不等于自己决定候选正确。

棋类把已知规则的后果经搜索转成训练目标；代码把候选程序经执行转成具体结果。它们都让明确、可重复的检查进入学习，使自生成经验可以反复用于改进下一次选择。

部署还会改变反馈分布

Performative prediction 研究模型部署后引起的数据变化。在强凸、联合平滑且分布变化足够弱等条件下，反复重训可以稳定，却未必达到部署后的最低风险。Miller 等据此研究先学习数据如何响应部署，再利用该响应优化的方法。反馈不仅检验当前模型，也可能随模型行动而改变。

参考文献

应用来源： Silver et al. (2018), A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play, Science; Le et al. (2022), CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning, NeurIPS.

[1] Singh et al. (2010). Intrinsically Motivated Reinforcement Learning: An Evolutionary Perspective. IEEE TAMD.

[2] Ma et al. (2024). Eureka: Human-Level Reward Design via Coding Large Language Models. ICLR; arXiv v2.

[3] Everitt et al. (2016). Self-Modification of Policy and Utility Function in Rational Agents. AGI; v1.

[4] Hay et al. (2012). Selecting Computations: Theory and Applications. UAI.

[5] Schmidhuber (2003/2006). Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. v5.

[6] Blackwell (1953). Equivalent Comparisons of Experiments. Ann. Math. Stat. 24:265–272.

[7] Klenske & Hennig (2016). Dual Control for Approximate Bayesian Reinforcement Learning. JMLR 17(127).

[8] Russo & Van Roy (2014). Learning to Optimize via Information-Directed Sampling. NeurIPS.

[9] Xu et al. (2020). A Theory of Usable Information under Computational Constraints. ICLR; v1.

[10] Abbe et al. (2022). The merged-staircase property: a necessary and nearly sufficient condition for SGD learning of sparse functions on two-layer neural networks. COLT.

[11] Schmidhuber (2009). Driven by Compression Progress: A Simple Principle Explains Essential Aspects of Subjective Beauty, Novelty, Surprise, Interestingness, Attention, Curiosity, Creativity, Art, Science, Music, Jokes. arXiv v2.

[12] Finzi et al. (2026). From Entropy to Epiplexity: Rethinking Information for Computationally Bounded Intelligence. 预印本 v2.

[13] Schick et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS; 核对 v1.

[14] Wang et al. (2024; 初稿 2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. TMLR; 核对 v2.

[15] Zhao et al. (2026). Memento 3: Model-Based Recursive Self-Improvement through Reflective Rulebooks. 预印本 v1.

[16] Gelman & Shalizi (2013). Philosophy and the practice of Bayesian statistics. BJMSP 66:8–38; 核对作者 v3.

[17] Andrychowicz et al. (2016). Learning to learn by gradient descent by gradient descent. NeurIPS; v2.

[18] Zoph & Le (2017). Neural Architecture Search with Reinforcement Learning. ICLR; v2.

[19] Wang et al. (2016/2017). Learning to reinforcement learn. arXiv:1611.05763v3.

[20] Zelikman et al. (2024). Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. COLM; v3.

[21] Zhang et al. (2025/2026). Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv v3.

[22] Zhang et al. (2026). Hyperagents. arXiv:2603.19461v1，预印本.

[23] Dohare et al. (2024). Loss of plasticity in deep continual learning. Nature 632:768–774; 方法核对作者 v3.

[24] Fudenberg & Levine (1993). Self-Confirming Equilibrium. Econometrica 61:523–545.

[25] Esponda & Pouzo (2016). Berk–Nash Equilibrium: A Framework for Modeling Agents with Misspecified Models. Econometrica; v4 含 2019 勘误.

[26] Fudenberg, Lanzani & Strack (2021). Limit Points of Endogenous Misspecified Learning. Econometrica 89:1065–1098.

[27] Perdomo et al. (2020). Performative Prediction. ICML.

[28] Miller, Perdomo & Zrnic (2021). Outside the Echo Chamber: Optimizing the Performative Risk. ICML.

[29] Shumailov et al. (2024). AI models collapse when trained on recursively generated data. Nature 631:755–759; 含 2025 修订.

[30] Gerstgrasser et al. (2024). Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv v2.

[31] Gao, Schulman & Hilton (2023). Scaling Laws for Reward Model Overoptimization. ICML.

[32] Arora et al. (2017). Generalization and Equilibrium in Generative Adversarial Nets (GANs). ICML.